

NEUROPLAST

A Self-Consolidating RL Agent

*Hybrid Spiking-Neural / Transformer Architecture · Hebbian-STDP + Gradient Descent · Sleep-Phase
Memory Consolidation*

Capstone Project Plan & Research Roadmap

Prepared as a working plan — architecture and scope open to revision as early prototypes report back

Contents

Contents	2
1. Topic of the Project	3
2. Problem Statement, Objective & Scope	3
2.1 Problem Statement	3
2.2 Objective & Research Questions	4
2.3 Scope	4
3. Hardware / Software Requirements	5
3.1 Compute	5
3.2 Software Stack	5
3.3 Benchmark Environments	6
4. Architecture, Process Flow & Timeline	6
4.1 System Overview: the Wake / Sleep Cycle	6
4.2 Three Candidate Architectures	7
4.3 Inside One Training Step: Two Learning Signals	8
4.4 Sleep-Phase Consolidation, in Detail	9
4.5 Timeline	10
4.6 Evaluation Metrics & Success Criteria	10
4.7 Risks & Mitigations	11
5. Usability / Applications	11
6. Related Work Consulted	12

(Right-click and choose "Update Field" in Word to populate page numbers.)

1. Topic of the Project

NeuroPlast is a deep reinforcement learning (RL) agent whose perception-to-action pipeline is deliberately modeled on how biological brains are believed to learn and consolidate memory, rather than following the now-standard “dense network trained end-to-end by backpropagation” recipe. Three ideas from neuroscience are combined into one system:

- A spiking neural network (SNN) core handles low-level perception using discrete, event-driven spikes instead of continuous activations — the same computational primitive real neurons use, and one that is far cheaper to run on neuromorphic hardware.
- A transformer sits on top of the SNN and plays the role of working memory: it attends over a short window of recent spiking activity to give the agent context, the way prefrontal circuits are believed to hold recent information “on-line” for decision-making.
- Learning combines two signals rather than one: standard gradient descent (via surrogate-gradient backprop through the spikes) alongside a local Hebbian/STDP (spike-timing-dependent plasticity) update that strengthens synapses purely from correlated pre/post-synaptic firing — no global error signal required.
- Periodically, the agent stops interacting with its environment and enters a “sleep” phase: it replays sampled past experience offline, consolidating what it has learned so that new tasks do not overwrite old ones — the artificial analogue of hippocampal replay during slow-wave and REM sleep.

The unifying research question underneath all of this is simple to state and genuinely open: do biologically-inspired learning and consolidation mechanisms buy an RL agent anything real — in sample efficiency, resistance to catastrophic forgetting, or energy efficiency — or are they scientifically interesting but practically redundant next to a well-tuned conventional deep RL baseline? NeuroPlast is built to answer that question empirically, not to assume the answer in advance.

2. Problem Statement, Objective & Scope

2.1 Problem Statement

Modern deep RL agents are powerful but have three well-known weaknesses that map directly onto properties biological brains seem to solve well:

- Catastrophic forgetting. A network trained sequentially on task B typically loses most of its competence on task A, because gradient updates for the new task overwrite weights the old task depended on. Standard fixes (experience replay, EWC-style regularization, generative replay) help but add substantial engineering overhead and rarely eliminate the problem.
- Energy cost of dense computation. Every unit in a conventional ANN fires on every forward pass, regardless of whether it carries useful information. Biological neurons spike sparsely and asynchronously, which is a large part of why the brain runs on roughly 20 watts. SNNs offer a path toward matching that sparsity in silico, but training them to competitive performance is still harder than training dense ANNs.
- Biological implausibility of backprop. End-to-end backpropagation requires exact, symmetric gradient information to flow backward through the whole network — something cortical circuits have no obvious mechanism for. Local rules like Hebbian learning and STDP are far more plausible biologically, but on their own they scale poorly to deep, high-performing networks.

None of these problems is solved by NeuroPlast alone — all three are active, unresolved research areas. What NeuroPlast contributes is a single, coherently-designed testbed where all three mechanisms (spiking perception,

hybrid local/global learning, sleep consolidation) are implemented together and evaluated against the same RL benchmarks and the same conventional baselines, so their individual and combined effects can actually be measured rather than assumed.

2.2 Objective & Research Questions

The objective is to design, build, and empirically evaluate a hybrid SNN-Transformer RL agent trained with a combined Hebbian/STDP + gradient-descent learning rule and a periodic sleep-replay consolidation cycle, and to compare it rigorously against conventional deep RL baselines on the same tasks. Four concrete, falsifiable research questions frame the work:

RQ	Question
RQ1	Does adding a local Hebbian/STDP term alongside gradient descent change sample efficiency or final return, compared to an otherwise identical architecture trained by backprop alone?
RQ2	Does a periodic sleep-replay consolidation phase reduce catastrophic forgetting across a sequence of tasks, relative to plain experience replay and an EWC-style baseline (measured via average forgetting and backward transfer)?
RQ3	Does the SNN encoder offer a better accuracy-vs-estimated-energy trade-off (spike sparsity as a proxy) than a same-capacity dense CNN encoder, and does that trade-off survive once the Hebbian/STDP term is added?
RQ4	(Stretch, paper-facing) Of the three candidate architectures in Section 4, which offers the best continual-learning robustness per unit of compute — and does the ranking change across task types?

These are written to be answerable even if the answer to some of them is “no” — a clean negative result (e.g., “STDP regularization does not improve sample efficiency, and here is the ablation showing why”) is a legitimate and publishable capstone/paper outcome, and is worth pre-registering as such before final experiments are run.

2.3 Scope

In scope

- Single-GPU, Colab-Pro-feasible benchmarks: lightweight pixel-based environments (e.g., MinAtar-style games, small procedurally generated tasks) and a small custom continual multi-task suite built from 3–5 such games in sequence.
- Implementing and comparing the three candidate architectures described in Section 4 (differentiable end-to-end, STDP-frontend + backprop-core, fully spiking transformer).
- A hybrid learning rule that mixes a local Hebbian/STDP update with the standard RL gradient, with the mixing weight as a tunable, swept hyperparameter.
- A sleep-phase replay/consolidation mechanism, evaluated against plain experience replay and at least one established continual-learning baseline (e.g., EWC or a simple generative-replay baseline).
- An energy proxy analysis (spike counts / estimated synaptic operations vs. dense-network FLOPs) — not physical power measurement.

Out of scope

- Deployment on physical neuromorphic hardware (Loihi 2, SpiNNaker, BrainScaleS) or real robots — noted only as future work.

- Multi-GPU / large-scale training, full Atari-57-style suites, or high-dimensional continuous-control humanoids.
- Claims of biological realism beyond algorithmic analogy — this is a bio-inspired engineering study, not a neuroscience model-validation study.

3. Hardware / Software Requirements

3.1 Compute

The plan is built around Colab Pro rather than a dedicated cluster, so every design choice below is chosen to survive that environment's constraints:

- GPU access is not guaranteed to be a specific model (T4 / L4 / A100 depending on availability and quota) — code should not assume a fixed VRAM budget; use mixed precision (torch.cuda.amp) and keep batch sizes / spike-simulation timesteps modest ($T = 4\text{--}16$) so runs fit comfortably on the smallest likely GPU.
- Sessions disconnect and time out (max $\sim 12\text{--}24\text{h}$, often less in practice). Checkpoint model + optimizer + replay-buffer state to Google Drive every few minutes and make the training loop fully resumable from the last checkpoint — treat this as a Day-1 requirement, not a nice-to-have.
- SNN simulation over T timesteps multiplies both compute and memory roughly $T\times$ relative to an equivalent static (non-spiking) network — this is the single biggest hidden cost in the whole project and the main reason the benchmark suite (3.3) is kept deliberately small early on.
- A local machine (even CPU-only) is enough for unit tests, debugging the STDP update math, and rendering diagrams / plots — reserve Colab GPU-hours for actual training runs.

3.2 Software Stack

Layer	Tool	Why
Core DL framework	PyTorch	Required by every SNN library below; native AMP + Colab support.
SNN library	snnTorch (primary); SpikingJelly for the fully-spiking transformer variant	snnTorch has the gentlest learning curve and best tutorials for surrogate-gradient LIF neurons; SpikingJelly's fused CUDA/CuPy kernels matter once Variant C's spiking attention gets expensive.
RL algorithm base	CleanRL single-file PPO / DQN	Easier to fork and splice a spiking encoder + hybrid learning rule into than a heavily-abstracted library like Stable-Baselines3.
Environments	Gymnasium + MinAtar; small Procgen subset	Pixel/grid observations, cheap to simulate, well-suited to Colab; MinAtar in particular is designed for fast iteration.
Continual benchmark	Custom 3–5-game MinAtar task sequence	Continual World / Meta-World need MuJoCo and heavier compute; a custom MinAtar sequence gets the same forgetting phenomena far more cheaply.

Experiment tracking	Weights & Biases (free tier)	Run comparison across the 3 architecture variants × hyperparameter sweeps needs first-class logging, not print statements.
Storage / checkpoints	Google Drive (mounted in Colab)	Survives session disconnects; pair with a resumable training loop.
Version control	GitHub (public or private repo)	Needed regardless — also becomes the code release if this turns into a paper.

Recommendation: start every new capability in `snnTorch` first (fastest to prototype), and only port the specific module to `SpikingJelly` if profiling shows it is the bottleneck — don't pay Norse/`SpikingJelly`'s steeper learning curve for modules that are not actually slow.

3.3 Benchmark Environments

- Single-task validation: 1–2 MinAtar games (e.g., Breakout-like, Space-Invaders-like) to sanity-check each architecture variant and each learning-rule combination before touching the harder continual setting.
- Continual-learning benchmark: a fixed sequence of 3–5 MinAtar-style games trained one after another, with periodic evaluation on all previously-seen games — this is what RQ2 and the forgetting metrics in Section 4 are measured on.
- Generalization probe (optional, time permitting): a small Procgen subset, to check whether conclusions hold when observations are procedurally varied rather than fixed.

4. Architecture, Process Flow & Timeline

This section is the core of the plan. Because the honest answer to “which architecture is best” is not known in advance, the plan is structured to prototype three genuinely different designs cheaply, compare them on a small proxy task, and only carry the strongest one (or two) forward into the full continual-learning integration — rather than committing to one design up front.

4.1 System Overview: the Wake / Sleep Cycle

At the top level, the agent alternates between two modes that never run at the same time:

- Wake phase: the usual online RL loop — observe, perceive (SNN), contextualize (Transformer), act (policy head), store the transition in an experience buffer.
- Sleep phase: triggered periodically (e.g., every N episodes, or at a task boundary), the agent stops acting, samples an interleaved mix of novel and familiar past experience from the buffer, replays it through the network offline, and updates weights via local STDP traces plus a distillation term that anchors old-task outputs — consolidating memory without any environment interaction.

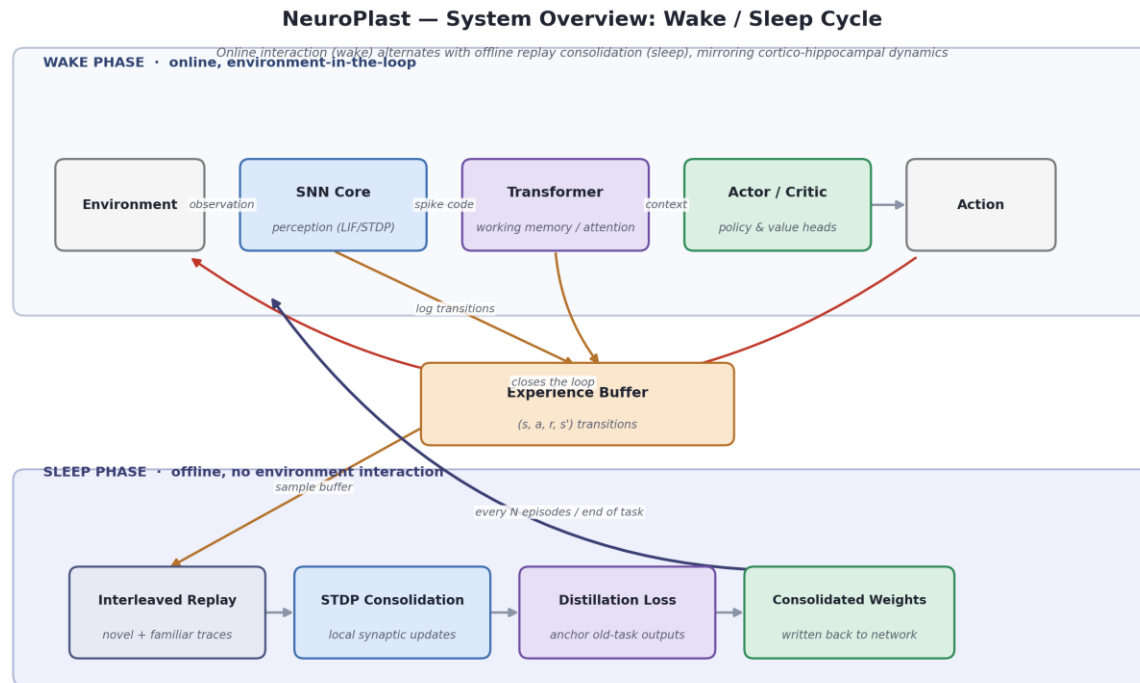


Figure 1. High-level wake/sleep cycle. The experience buffer is the hinge between the two phases.

4.2 Three Candidate Architectures

All three variants share the same input/output contract (observation in, action out) so they can be swapped behind an identical training harness and compared fairly.

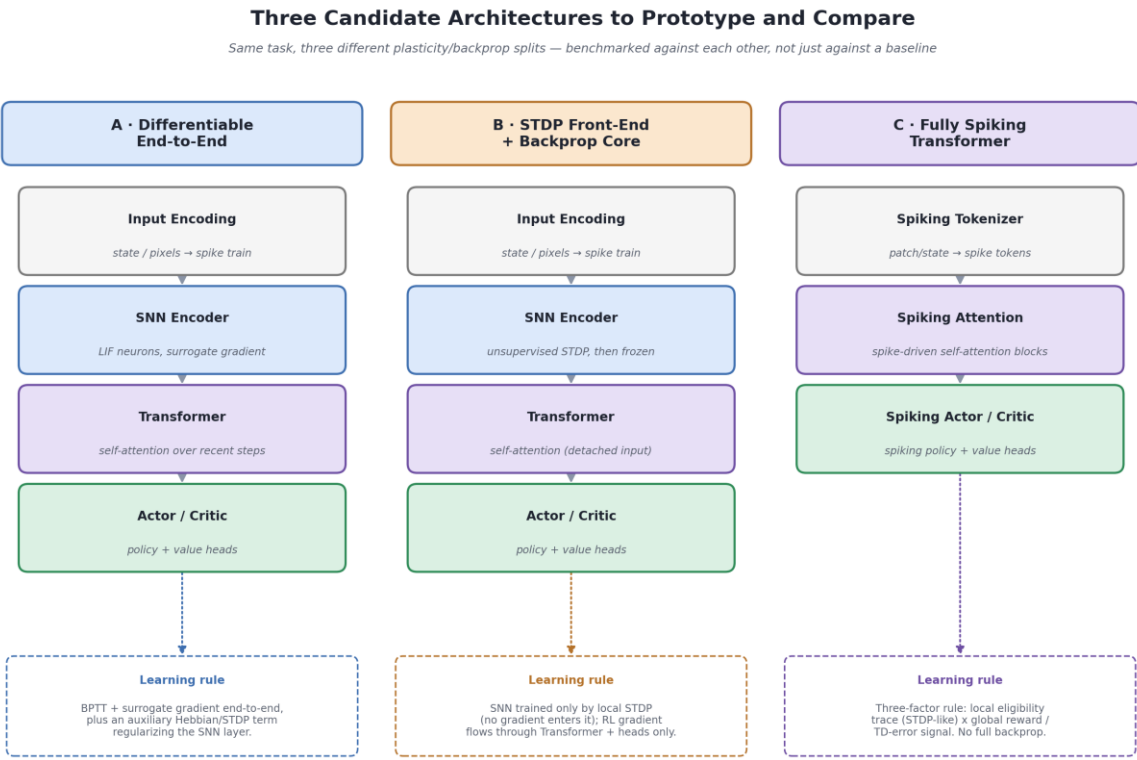


Figure 2. Variants A, B, and C — same task, three different plasticity/backprop splits.

	A · Differentiable end-to-end	B · STDP front-end + backprop core	C · Fully spiking transformer
Biological plausibility	Lowest of the three (global gradient still flows through the SNN)	Higher (SNN weights never see a global gradient)	Highest (three-factor local rule throughout)
Expected engineering difficulty	Lowest — closest to a standard SNN-RL pipeline	Medium — two separate training regimes to keep in sync	Highest — spiking attention is still niche, thin tooling
Expected training stability	Best-understood (surrogate-gradient BPTT is well studied)	Moderate — frozen/local front-end can under-fit	Riskiest — reward-modulated local rules are less mature
Role in the plan	Default “spine” — build and validate this first	Cheap variant to fork from A once the spine works	Stretch goal — only pursued if A/B finish with time to spare

Decision point (end of Phase 3, Section 4.5): run all three variants on the cheap single-task proxy benchmark only, then commit the remaining budget to whichever one (or two) shows the best combination of stability and performance, rather than integrating all three into the full continual-learning pipeline. This is the single most important scope-control decision in the whole plan.

4.3 Inside One Training Step: Two Learning Signals

Whichever variant is chosen, the mechanism that makes NeuroPlast “hybrid” is the same: on every step, the SNN synapses receive two updates, computed from two different signals, and combined with a tunable mixing weight.

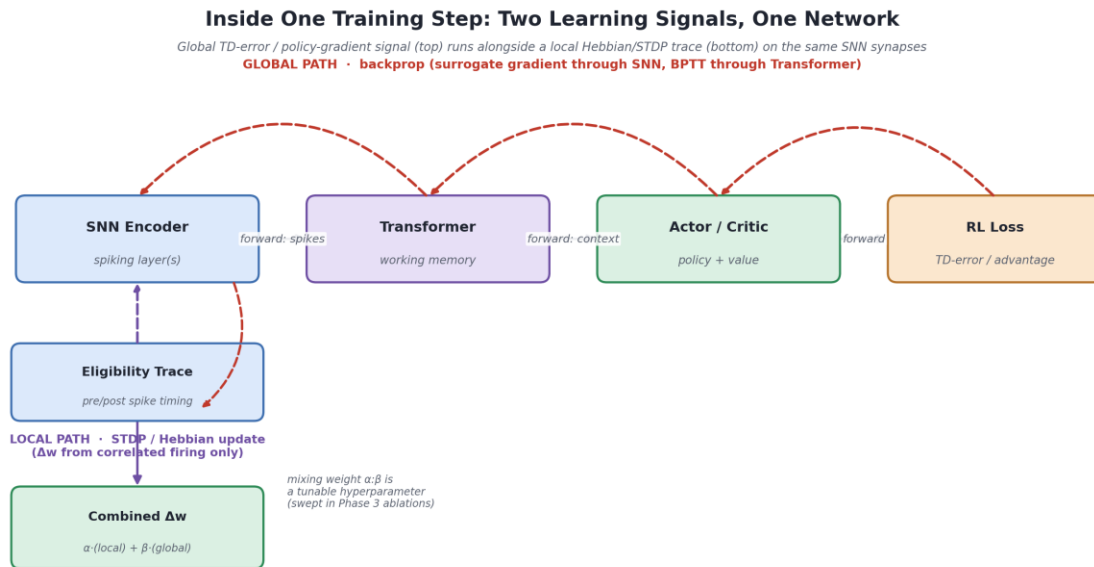


Figure 3. The global (backprop / TD-error) path and the local (STDP / Hebbian) path update the same synapses; a mixing weight $\alpha \cdot \beta$ controls their relative influence, and is swept as a hyperparameter.

Concretely: the global path back-propagates the RL loss (advantage-weighted policy gradient, or TD-error for value-based methods) through the Transformer and, via a surrogate gradient, through the spiking layers. The local path accumulates an eligibility trace from pre/post-synaptic spike timing at each SNN synapse, independent of any downstream loss. The two contributions are combined as $\Delta w = \alpha \cdot (\text{local STDP term}) + \beta \cdot (\text{global gradient term})$, with $\alpha \cdot \beta$ swept during Phase 3 rather than fixed by intuition.

4.4 Sleep-Phase Consolidation, in Detail

The sleep phase (right-hand side of Figure 1) is deliberately built in two stages of increasing sophistication, so there is always a working fallback:

- Baseline version (build first): sample a batch from the experience buffer, replay it through the network offline, and apply a standard distillation/regularization loss that anchors current outputs close to what the network produced on that data before the most recent task — conceptually similar to Elastic Weight Consolidation or simple rehearsal.
- Neuro-inspired version (build second, if time allows): interleave replay of novel (just-learned) and familiar (older) experience within the same sleep phase, rather than replaying them in separate blocks — mirroring recent findings that biological slow-wave sleep interleaves novel and familiar memory traces within individual slow oscillations to avoid interference. This is a direct, testable hypothesis for RQ2, not just a design flourish.

Both versions are evaluated against the same forgetting metrics (Section 4.6) so the marginal benefit of the more elaborate, biologically-motivated version can be measured rather than assumed.

4.5 Timeline

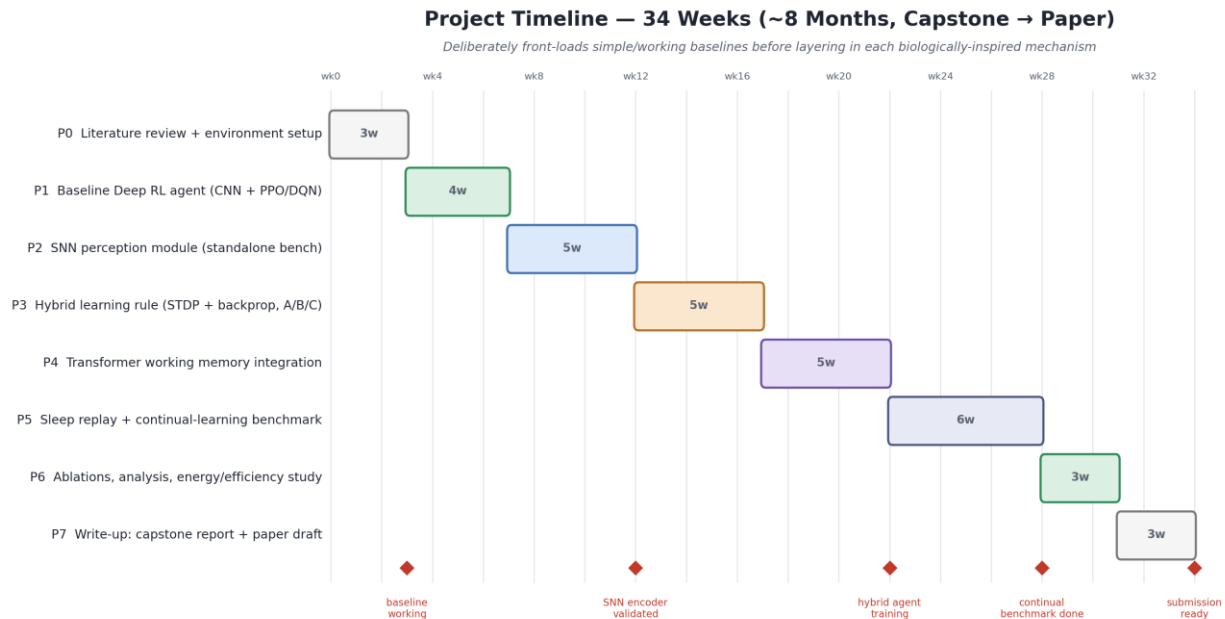


Figure 4. 34-week phased plan (~8 months), leaving buffer inside a 6+ month timeline for slip and a paper-writing pass.

Phase	Focus	Exit criterion
P0 (wk 1–3)	Literature review; Colab/Drive/W&B setup; reproduce a known SNN-RL result as a sanity check.	Environment + tooling runs end-to-end on a toy task.
P1 (wk 3–7)	Conventional CNN + PPO/DQN baseline on the chosen single-task benchmark.	Baseline reaches a documented, reproducible score.
P2 (wk 7–12)	SNN perception module, benchmarked standalone against the CNN encoder (accuracy + spike-sparsity).	SNN encoder trains stably and its efficiency trade-off is measured (RQ3).
P3 (wk 12–17)	Implement the hybrid STDP+backprop rule; prototype Variants A/B/C on the cheap proxy task; make the decision-point call (4.2).	One (or two) variants selected, with a documented reason.
P4 (wk 17–22)	Integrate the Transformer working-memory layer on top of the selected variant(s); full agent trains on the single-task benchmark.	Full agent matches or approaches baseline on a single task.
P5 (wk 22–28)	Sleep-replay consolidation (baseline, then neuro-inspired version); run the full continual multi-task benchmark against EWC / plain-replay baselines.	Forgetting metrics computed for all conditions (RQ2).
P6 (wk 28–31)	Ablations across α ; β , STDP time constants, sleep frequency; energy-proxy analysis; seed variance runs.	All RQs answered with supporting numbers, including any negative results.
P7 (wk 31–34)	Write-up: capstone report, and (if results warrant) a workshop-paper draft.	Report submitted; repo public with a README reproducing key results.

4.6 Evaluation Metrics & Success Criteria

- Task performance: episodic return curves, per task, per architecture variant, per learning-rule condition.
- Sample efficiency: environment steps to reach a fixed performance threshold.
- Forgetting: standard continual-RL metric — average drop in performance on task i after training on tasks $i+1 \dots n$, relative to performance right after task i was learned.
- Backward/forward transfer: whether learning a later task helps or hurts performance on earlier/later tasks.
- Energy proxy: mean spike count / estimated synaptic operations per forward pass, vs. dense-network FLOPs at matched parameter count.
- Statistical reliability: ≥ 3 –5 random seeds per condition, reported with variance — single-seed comparisons are not treated as conclusive.

4.7 Risks & Mitigations

Risk	Likelihood	Mitigation
Surrogate-gradient SNN training is unstable / spikes vanish	Medium–High	Use well-tested LIF neurons with adaptive thresholds; start shallow; borrow normalization tricks from recent SNN-RL stabilization work.
Colab disconnects lose training progress	High	Checkpoint to Drive every few minutes; resumable training loop from Day 1; keep individual runs short where possible.
Three architecture variants triple the engineering load	High	Build Variant A as a shared “spine”; reuse >80% of code for B and C; timebox exploration and use the Phase 3 decision point to cut scope.
Hybrid-rule hyperparameters (α ; β , STDP time constants) create a huge search space	Medium	Small grid/random search on the cheap proxy task before scaling any sweep to the full benchmark.
Core hypothesis turns out negative (bio-inspired rules don’t help)	Medium	Pre-register RQs and metrics; a rigorous negative result with clean ablations is still a valid capstone/workshop-paper contribution.
Exactly reproducing sleep-replay literature setups is nontrivial	Medium	Ship the simple distillation-based sleep phase first (4.4); add interleaved novel/familiar replay only once that baseline works.

5. Usability / Applications

Even scoped down to Colab-sized benchmarks, the mechanisms NeuroPlast tests are directly relevant to several real application areas — which is worth stating plainly in a capstone report, since it is the difference between “a demo” and “a contribution”:

- Edge and embedded robotics: SNN perception is a natural fit for battery-powered or embedded agents, and its sparsity maps directly onto emerging low-power neuromorphic accelerators, making the accuracy-vs-energy results in RQ3 practically relevant, not just academic.

- Lifelong / continual-learning agents: any system that must keep learning after deployment without forgetting earlier skills — a household or warehouse robot picking up new tasks, a personal assistant adapting to a new user’s preferences, a recommendation system tracking drifting interests — faces exactly the catastrophic-forgetting problem RQ2 targets.
- More robust lifelong agents, generally: mechanisms that reduce silent regression on previously-learned (and previously-verified) behavior are relevant to building agents whose behavior stays predictable as they keep learning — a modest but real practical motivation, distinct from any claim about AI safety at large.
- Research contribution: a controlled, same-codebase comparison of three plausibility/performance trade-off points (Section 4.2) adds a genuinely useful data point to the growing neuro-AI literature bridging neuroscience and reinforcement learning — useful independent of which architecture wins.
- Reusable tooling: the benchmark harness (baselines + all three variants + continual-task suite + forgetting metrics) is valuable on its own as an open-source artifact other students or researchers could build on.

6. Related Work Consulted

Directly informing the design choices above (paraphrased here; full citations in the repository bibliography):

- Golden, Saxena, González, Delanois, Kilianski, McNaughton & Bazhenov (2025, bioRxiv) — propose that slow-wave sleep interleaves replay of novel and familiar memory traces within individual slow oscillations to avoid interference; direct inspiration for the neuro-inspired sleep-phase design in 4.4.
- A Nature Communications study on sleep-like unsupervised replay — shows an unsupervised, REM-like replay phase combined with a standard rehearsal method (iCaRL) reduces catastrophic forgetting in ANNs; a template for the baseline sleep-phase design.
- “Spiking Decision Transformers” (arXiv, 2025) — combines local plasticity, phase-coding, and dendritic routing in a low-power spiking sequence-control model; closest existing prior art to Variant C.
- “CaRe-BN” (ICLR 2026) — a normalization technique for stabilizing spiking neural networks specifically in the RL setting; directly relevant to the training-stability risk in Section 4.7.
- “Adaptive Surrogate Gradients for Sequential RL in SNNs” (arXiv, 2025) and related work on population-coded and dynamics-aware spiking RL agents — inform the SNN-encoder design in Variant A.
- SpikingJelly (Fang et al.), Norse (Pehle & Pedersen), and snnTorch (Eshraghian et al.) — the three mainstream PyTorch-based SNN libraries surveyed in Section 3.2; snnTorch and SpikingJelly are the two carried forward into the stack.

These are starting points, not a completed literature review — Phase 0 (Section 4.5) is explicitly budgeted to expand this list before any code is written.